Infosys Gen AI LLMOps Engineer — Interview Q&A

Based on Your Market Analyst Pro Codebase

🔷 SECTION 1: LangGraph & Agentic Architecture

Q1. Walk me through your LangGraph state machine. Why did you choose a `router → agent → validator → writer` pattern instead of a simple ReAct loop?

Answer:

A simple ReAct (Reason + Act) loop works well for single-tool, single-step tasks. But my use case — a Market Analyst that queries FAISS, SQLite, CSV files, and live web search — needed more control over quality and routing.

Here's why each node exists:

Node	Role
`router`	Classifies intent (local/web/hybrid/general) from user text
`agent`	LLM with MCP tools bound; decides which tool to call
`tools`	ToolNode that executes the actual MCP tool
`validator`	Checks if tool output is weak/empty and decides retry or proceed
`writer`	Final LLM pass to produce a clean, user-facing response

Why not plain ReAct?

- ReAct loops can hallucinate confidently even with empty tool results.
- My `validator_node` explicitly checks for weak markers like `"No local records found"` before allowing the final answer.
- The `writer` node separates the "reasoning" LLM call from the "presentation" LLM call — this avoids tool-call artifacts leaking into the final response.
- The `router` node reduces unnecessary tool calls by routing intent upfront.

Q2. How does `add_messages` in your `GraphState` TypedDict differ from a plain list?

Answer:

`add_messages` is a **reducer function** provided by LangGraph.

```python
class GraphState(TypedDict, total=False):
    messages: Annotated[list, add_messages]
```

In LangGraph, every node returns a **partial state update** — it doesn't return the full state. Without a reducer, returning `{"messages": [new_msg]}` would **replace** the entire messages list.

`add_messages` acts as a merge strategy: it **appends** new messages to the existing list instead of overwriting. It also handles deduplication by message ID.

Plain list behavior:

```python
```

```
# Node returns {"messages": [new_msg]}
# State becomes: [new_msg]  ← old messages lost!
```

With add_messages:

```python
# State becomes: [old_msg_1, old_msg_2, new_msg]  ← preserved
```

This is critical for multi-turn conversation history and for the checkpointer (`AsyncSqliteSaver`) to persist the full thread correctly.

Q3. Your `route_after_agent` calls `tools_condition` — what does it check internally?

Answer:

`tools_condition` is a LangGraph prebuilt conditional function. Internally it inspects the **last AI message** in the state and checks whether it contains `tool_calls` in its metadata.

```python
def route_after_agent(state: GraphState) -> Literal["tools", "validator"]:
    result = tools_condition(state)
    if result == "tools":
        return "tools"
    return "validator"
```

When the LLM decides to call a tool, it returns an `AIMessage` with a non-empty `tool_calls` list. `tools_condition` returns `"tools"` in that case, or `"__end__"` otherwise.

I wrapped it because `tools_condition` returns `"__end__"` when no tool is called, but I needed it to route to `"validator"` instead — my graph has no direct END from the agent.

Q4. Your `validator_node` retries once on weak results. What are the trade-offs vs. letting the LLM self-correct?

Answer:

My approach — explicit validator node:

✅ Deterministic: I define exactly what "weak" means (specific string markers). ✅ Cost-controlled: Max 1 retry, not an infinite LLM loop. ✅ Transparent: Easy to debug — I can log `validation_status`. ❌ Brittle: If a tool returns a new error format I haven't anticipated, the validator misses it.

LLM self-correction approach:

✅ Flexible: The LLM can reason about any failure mode.
❌ Expensive: Each self-correction is a full LLM call.
❌ Unpredictable: The LLM might decide the empty result is fine and proceed.

In production I'd combine both: the validator for deterministic guardrails + a critic LLM prompt for semantic quality checks (e.g., RAGAS faithfulness score).

Q5. What is `AsyncSqliteSaver` doing, and why initialize it inside `lifespan`?

Answer:

`AsyncSqliteSaver` is LangGraph's built-in **checkpointer** — it persists the full graph state (messages, route, retry_count, etc.) to a SQLite database after every node execution.

1. **Thread continuity** — the same `thread_id` resumes conversation history across API calls.
2. **Crash recovery** — if the server restarts, state is not lost.
3. **Time-travel debugging** — LangGraph can replay from any checkpoint.

Why inside `lifespan`?

`AsyncSqliteSaver.from_conn_string()` is an `async context manager` — it opens and manages an async SQLite connection. It must live for the entire app lifetime. Initializing it at module level would try to open an async connection outside any event loop, which raises a `RuntimeError`. The FastAPI `lifespan` context manager runs inside the asyncio event loop, so it's the correct place.

🔷 SECTION 2: RAG Pipeline & Vector Search

Q6. What chunk size and overlap did you choose and why?

Answer:

For financial/corporate documents (earnings reports, market notes), I used:

- **Chunk size: 512 tokens** — large enough to preserve context around a financial metric, small enough to stay within embedding model limits.
- **Overlap: 50 tokens** — ensures sentences split at a chunk boundary don't lose context.

The rule of thumb: chunk size should match the **granularity of your expected queries**. For "What was Accenture Q2 revenue?" — a 512-token chunk around a financial paragraph is

ideal. For code or tabular data, smaller chunks (128–256) work better.

Q7. Your retriever uses `k=5`. How would you tune k for precision vs. recall?

Answer:

- **Low k (1–3):** High precision, low recall. Good when your index is clean and queries are specific. Risk: missing the right chunk if ranking is off.
- **High k (5–10):** High recall, lower precision. Good for broad queries. Risk: irrelevant chunks dilute the context window and confuse the LLM.

Tuning strategy:

1. Use RAGAS `context_recall` and `context_precision` metrics on a golden QA dataset.
2. Start with k=5, measure, then adjust.
3. For production: use **MMR (Maximum Marginal Relevance)** retrieval — `search_type="mmr"` in LangChain — to balance relevance and diversity.

In my case k=5 was a reasonable default for a mixed corpus of earnings docs and market notes.

Q8. FAISS vs. Pinecone — what changes architecturally in production?

Answer:

Aspect	FAISS (my current setup)	Pinecone (production)
Hosting	Local file on disk	Managed cloud service
Scaling	Single machine	Horizontally scalable
Updates	Re-build index for new docs	Upsert vectors via API
Filtering	Manual post-filter	Metadata filtering built-in
Cost	Free	Per-vector pricing

What changes in code:

```python
# FAISS
vector_db = FAISS.load_local(FAISS_PATH, embeddings)

# Pinecone
from langchain_pinecone import PineconeVectorStore
vector_db = PineconeVectorStore(index_name="market-analyst", embedding=embeddin
```

The `retriever` interface stays identical — LangChain abstracts the underlying store. The main architectural change is the **ingestion pipeline**: FAISS requires a full re-index; Pinecone supports incremental upserts, which is critical for live market data.

Q9. `allow_dangerous_deserialization=True` — what's the risk and mitigation?

Answer:

FAISS uses Python's `pickle` to serialize index data. Pickle can execute arbitrary code during deserialization — a maliciously crafted FAISS index file could run code on your server.

Risk: If the `faiss_index/` directory is writable by an untrusted source (e.g., user-uploaded files, compromised CI pipeline), an attacker could replace the index with a malicious pickle.

Mitigations in production:

1. Store the FAISS index in a **read-only, access-controlled location**.
2. **Checksum validation** — store a SHA256 hash of the index file at build time, verify before loading.
3. Move to **Pinecone or Chroma** which don't use pickle serialization.
4. Run the loading process in an **isolated container** with no network access.

Q10. How does `gemini-embedding-001` compare to OpenAI embeddings?

Answer:

Property	`gemini-embedding-001`	`text-embedding-3-small` (OpenAI)
Dimensions	768	1536 (default), scalable
Multilingual	Strong	Strong
Cost	Google AI pricing	OpenAI pricing
Task types	Supports `task_type` param	Fixed

Gemini embeddings support a `task_type` parameter (`RETRIEVAL_DOCUMENT` vs `RETRIEVAL_QUERY`) which can improve retrieval quality by using asymmetric embeddings — document and query embeddings are optimized differently. OpenAI's `text-embedding-3` models support dimension reduction, useful for cost/latency optimization. For my use case (English financial docs), both perform comparably; I chose Gemini for its free tier during development.

🔷 SECTION 3: LLM Deployment & API Design

Q11. How does `astream_events` with `version="v2"` differ from `astream`?

Answer:

`astream` yields **node-level state updates** — you get the full state dict after each node completes. It's coarse-grained.

`astream_events` yields **fine-grained events** at every level of execution:

Event	When fired
`on_chat_model_stream`	Each LLM token chunk
`on_tool_start`	Tool begins executing
`on_tool_end`	Tool finishes, returns result
`on_chain_start/end`	Each LangChain chain

Why I chose `astream_events`: It lets me stream LLM tokens to the React frontend in real-time AND emit tool progress events — impossible with `astream`.

Q12. What happens if the MCP server is down at boot?

Answer:

In my code:

```python
try:
    mcp_tools = await mcp_client.get_tools()
except Exception as e:
    print(f"--- MCP LOAD FAILED: {e} ---")
    mcp_tools = []
```

If MCP is down, `mcp_tools` is empty. Then:

```python
llm_with_tools = llm.bind_tools(mcp_tools)  # only set if tools loaded
active_llm = llm_with_tools if llm_with_tools is not None else llm
```

The agent falls back to the base LLM without tools — it can still answer general questions but can't query FAISS or web. The app stays alive.

Production improvement: Add a health check endpoint for the MCP server and implement a **retry with backoff** on startup, plus a `/health` endpoint that reports tool availability.

Q13. `X-Accel-Buffering: no` **— why is it critical?**

Answer:

Nginx by default **buffers upstream responses** — it waits to accumulate data before forwarding to the client. For normal HTTP responses this improves performance. But for SSE (Server-Sent Events), buffering breaks the streaming entirely — the client receives nothing until the buffer fills or the connection closes.

`X-Accel-Buffering: no` tells Nginx to **pass each SSE chunk to the client immediately** without buffering.

Without it: the user sees a loading spinner for 30 seconds, then all tokens appear at once — defeating the purpose of streaming.

Also needed: `Cache-Control: no-cache` to prevent any intermediate proxy from caching the stream.

Q14. How would you handle CORS for multi-environment deployments?

Answer:

Instead of hardcoding `localhost:3000`, I'd use environment variables:

```
python
```

```
import os

ALLOWED_ORIGINS = os.getenv("ALLOWED_ORIGINS", "http://localhost:3000").split("

app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Then in `.env` files per environment:

```
# .env.development
ALLOWED_ORIGINS=http://localhost:3000

# .env.production
ALLOWED_ORIGINS=https://marketanalyst.myapp.com,https://app.myapp.com
```

For Cloud Run / GCP, inject as environment variables in the deployment config.

🔷 SECTION 4: MCP Integration

Q15. Explain the SSE transport in MCP — why SSE over stdio?

Answer:

MCP supports three transports:

Transport	Use Case
`stdio`	Local CLI tools, subprocess communication
`SSE`	Network-accessible server, async streaming
`HTTP`	Simple request/response, no streaming

I chose **SSE** because:

1. My MCP server runs as a separate process on `127.0.0.1:8000` — it's a networked service, not a subprocess.
2. The `ctx.report_progress()` calls in `mcp_search_the_web` need to push events **back to the caller** in real time — SSE's persistent connection enables this.
3. `MultiServerMCPClient` in LangChain natively supports SSE transport with async tool calls.

stdio would only work if the MCP server was launched as a child process of the FastAPI app — not suitable for a separately deployed service.

Q16. How does `ctx.report_progress()` flow from MCP server to React frontend?

Answer:

The flow is a chain of SSE connections:

In practice, `ctx.report_progress()` sends progress over the MCP SSE connection back to `MultiServerMCPClient`. LangGraph captures this as `on_tool_start` / `on_tool_end` events. My `event_generator` in FastAPI intercepts these and forwards them as SSE chunks to the React `EventSource`.

Q17. Why fake progress in `tool_progress_map` instead of relying on real MCP events?

Answer:

Real `ctx.report_progress()` events from MCP travel over a separate internal SSE connection (MCP client ↔ MCP server). LangGraph's `astream_events` does **not** surface MCP progress as granular sub-events — it only emits `on_tool_start` (before the tool runs) and `on_tool_end` (after it completes).

So during tool execution, the frontend would see nothing — a dead period of 5–15 seconds for a web search. The fake `tool_progress_map` simulates intermediate progress steps with `asyncio.sleep(0.3)` delays to keep the UI animated and responsive.

This is a deliberate UX trade-off: perceived performance > technical accuracy.

Q18. What does `langchain_mcp_adapters` do when calling `get_tools()`?

Answer:

`MultiServerMCPClient.get_tools()` does the following:

1. Connects to the MCP server via SSE.
2. Calls the MCP `tools/list` protocol method to fetch the tool manifest.
3. For each tool, it creates a LangChain `StructuredTool` object with:
    - `name` → MCP tool name (e.g., `mcp_search_the_web`)
    - `description` → from the `@mcp.tool()` docstring
    - `args_schema` → Pydantic model generated from the tool's type annotations
    - `coroutine` → async wrapper that calls the MCP tool via SSE

This lets LangGraph's `ToolNode` treat MCP tools identically to any native LangChain tool — it calls `.invoke()` and gets back a `ToolMessage`.

🔷 SECTION 5: LLMOps & Production

Q19. How would you build systematic guardrails using RAGAS or DeepEval?

Answer:

My current validator checks for string markers — it's a heuristic. A production system needs semantic evaluation.

Using RAGAS:

```
python
```

```python
from ragas.metrics import faithfulness, answer_relevancy, context_recall
from ragas import evaluate

# After each response, evaluate asynchronously
dataset = {
    "question": [user_query],
    "answer": [final_answer],
    "contexts": [retrieved_chunks],
    "ground_truth": [expected_answer]  # for offline eval
}
result = evaluate(dataset, metrics=[faithfulness, answer_relevancy])
```

In production pipeline:

1. Log every (query, retrieved_context, answer) tuple.
2. Run RAGAS `faithfulness` score — catches hallucinations where the answer contradicts the retrieved context.
3. If faithfulness < 0.7, trigger a re-generation with a stricter prompt.
4. Use DeepEval's `HallucinationMetric` for real-time guardrails in the API path.

Q20. How would you implement prompt versioning and A/B testing?

Answer:

Prompt versioning: Store prompts in a config file or database, not hardcoded:

```python
```

```
# prompts.yaml
agent_v1: "You are a Market Analyst. Use tools for factual lookup..."
agent_v2: "You are a senior Market Analyst with 20 years experience..."

# Load at startup
ACTIVE_PROMPT = os.getenv("AGENT_PROMPT_VERSION", "agent_v1")
system_prompt = prompts[ACTIVE_PROMPT]
```

Use **LangSmith** for prompt versioning — it stores prompt templates with version tags and links them to traces.

A/B testing:

python

```
import random
variant = "v1" if random.random() < 0.5 else "v2"
system_prompt = prompts[f"agent_{variant}"]
# Log variant with thread_id for analysis
```

Measure: response quality score, tool call count, latency, user satisfaction signals.

Q21. When would you increase `temperature=0` in a market analyst context?

Answer:

`temperature=0` means **deterministic, greedy decoding** — the LLM always picks the highest-probability token. This is correct for:

- Factual lookups (trade history, signal data)
- Structured output (JSON formatting)

- Tool selection

I would **increase temperature (0.3–0.7)** for:

- **Report generation** — slightly varied phrasing makes reports feel less robotic.
- **Brainstorming** — generating multiple trading hypotheses.
- **Creative summarization** — synthesizing qualitative market commentary.

Risk at higher temperature in finance: The LLM might invent specific numbers (hallucinate "$18.5B revenue" vs the real "$18.0B"). Always keep tool-calling and retrieval nodes at temperature=0; only increase for the final `writer_node`.

Q22. How would you monitor LLM latency and cost per query?

Answer:

LangSmith tracing (already integrates with LangGraph):

```python
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "..."
```

Every graph run is automatically traced — you get per-node latency, token counts, and cost estimates.

Custom metrics with Prometheus:

```python
```

```python
from prometheus_client import Histogram, Counter

llm_latency = Histogram("llm_call_duration_seconds", "LLM call latency")
llm_tokens = Counter("llm_tokens_total", "Total tokens used")

# Wrap LLM call
with llm_latency.time():
    response = await llm.ainvoke(messages)
llm_tokens.inc(response.usage_metadata["total_tokens"])
```

Alerting thresholds for Groq/LLaMA:

- Latency > 5s per query → alert
- Cost > $0.05 per query → alert
- Token usage spike > 2x baseline → alert

Q23. How would you design exponential backoff + fallback-to-base-model?

Answer:

```python
```

```python
import asyncio

async def agent_node_with_backoff(state: GraphState):
    retry_count = state.get("retry_count", 0)

    # Exponential backoff: 1s, 2s, 4s
    if retry_count > 0:
        wait = 2 ** (retry_count - 1)
        await asyncio.sleep(wait)

    # Fallback: use base LLM after 2 retries
    if retry_count >= 2:
        active_llm = llm  # no tools
        system_prompt += "\nNote: Tool access unavailable. Answer from your kno
    else:
        active_llm = llm_with_tools

    response = await active_llm.ainvoke([SystemMessage(content=system_prompt)]
    return {"messages": [response]}
```

Additionally, implement **circuit breaker** pattern: if MCP tools fail 5 times in 60 seconds, disable them entirely for 5 minutes and serve from base LLM only.

🔷 SECTION 6: Scheduler & Background Jobs

Q24. How would you make APScheduler timezone-aware for IST on a UTC cloud VM?

Answer:

```python
from apscheduler.schedulers.asyncio import AsyncIOScheduler
import pytz

scheduler = AsyncIOScheduler(timezone=pytz.timezone("Asia/Kolkata"))
scheduler.add_job(
    daily_trade_job,
    'cron',
    day_of_week='mon-fri',
    hour=9,
    minute=15
    # No UTC conversion needed — scheduler handles it
)
```

Without specifying timezone, APScheduler uses the system timezone. On a Google Cloud Run container (UTC), `hour=9` would fire at 9 AM UTC = 2:30 PM IST — completely wrong for market open.

Verification: Always log `datetime.now(pytz.timezone("Asia/Kolkata"))` at job start to confirm correct local time.

Q25. Why is `db_session.remove()` necessary in the finally block?

Answer:

`db_session` is a **SQLAlchemy scoped session** — it returns a thread-local (or async-context-local) session from a connection pool.

In an `asyncio` background task, if you don't call `.remove()`:

1. The session stays open, holding a **database connection** from the pool.

2. On the next job run, a new session is created — but the old one may still hold a lock.
3. Over time, the connection pool is **exhausted**, and new requests hang waiting for a connection.

`db_session.remove()` returns the connection back to the pool and clears the session registry. The `finally` block guarantees this happens even if an exception is raised mid-execution.

🔷 SECTION 7: Quick-Fire Conceptual Answers

Q: What is the difference between `ainvoke` and `astream_events`?

`ainvoke` is a single async call that returns the complete result after full execution — no intermediate output. `astream_events` is an async generator that yields events at every step of execution (token-by-token, tool-by-tool). Use `ainvoke` for batch processing; use `astream_events` for real-time streaming UIs.

Q: What does `bind_tools` do to an LLM?

`llm.bind_tools(tools)` modifies the LLM's API call to include a `tools` parameter with the JSON schema of each tool. The LLM can then return an `AIMessage` with `tool_calls` populated — indicating which tool to call and with what arguments. It does not change the LLM weights; it only changes the API payload.

Q: Why use `TypedDict` for graph state?

`TypedDict` provides type hints for LangGraph's state schema — it tells LangGraph which keys exist, their types, and (via `Annotated`) which reducer functions to apply. It also enables IDE autocomplete and static type checking with `mypy`. The `total=False` parameter makes all keys optional, so nodes can return partial state updates without KeyErrors.

Q: What is a checkpoint in LangGraph?

A checkpoint is a snapshot of the full graph state saved to persistent storage (SQLite in my case) after each node execution. It stores: all messages, current node, retry_count, validation_status, and thread_id. Checkpoints enable: (1) conversation continuity across API calls, (2) crash recovery, (3) LangGraph Studio time-travel debugging where you can replay from any past state.

Q: How does FAISS similarity search work internally?

FAISS converts both the query and stored documents into dense vector embeddings (768-dimensional floats in my case with Gemini). It then computes **cosine similarity** (or L2 distance, depending on index type) between the query vector and all stored vectors. The `IndexFlatL2` index (FAISS default) does exact brute-force search. For large corpora, `IndexIVFFlat` uses inverted file clustering — it first identifies the nearest cluster centroids, then searches only within those clusters, trading slight accuracy for major speed gains. My `k=5` means it returns the 5 most similar document chunks.

Prepared specifically for Narayanan Selvaraj — Infosys Gen AI LLMOps Engineer Interview
Role: Gen AI LLMOps Engineer | Location: Bangalore | Job ID: INFSYS-EXTERNAL-245907